



BÁO CÁO BÀI THỰC HÀNH SỐ 3

Lập trình Assembly cơ bản (tiếp theo)

Môn học: NT209 - Lập trình Hệ thống

Sinh viên thực hiện	Lê Ngọc Phương Thảo - 23521467 Bùi Nguyễn Minh Anh - 24520086
Thời gian thực hiện	23/04/2026 – 04/05/2026
Số câu đã hoàn thành	4/4

NỘI DUNG BÁO CÁO

C.1 Chương trình kiểm tra tính tăng dần của các chữ số trong số có 4 chữ số

Input: Một số nguyên a có 4 chữ số ($1000 \leq a \leq 9999$)

Output: Xuất ra màn hình nhận định: **"Tang dan"** nếu các chữ số trong a tăng dần, ngược lại xuất ra **"Khong tang dan"**

Mã nguồn (Source code):

```
.section .data
input_str: .string "Enter a number (4-digits): "
input_len = . -input_str

newline: .byte 10

inc_str: .string "Tang dan"
inc_len = . -inc_str

dec_str: .string "Khong tang dan"
dec_len = . -dec_str

.section .bss
.lcomm input, 4

.section .text
.globl _start
_start:
    # Xuất chuỗi yêu cầu nhập input
    movl $4, %eax           # sys_write
    movl $1, %ebx           # stdout
    movl $input_str, %ecx    # chuỗi in
    movl $input_len, %edx    # độ dài chuỗi
    int $0x80

    # Nhập chuỗi input
    movl $3, %eax           # sys_read
    movl $0, %ebx           # stdin
    movl $input, %ecx       # lưu giá trị nhập biến input
    movl $5, %edx           # độ dài tối đa của chuỗi input
    int $0x80
```

```

# Khởi tạo biến chạy vị trí để so sánh
leal 3(%ecx), %ecx

_for_loop:
# So sánh vị trí current (ecx) và vị trí gốc của input
cmpl $input, %ecx
jle _inc          # nếu ecx < input
                  # --> duyệt xong và nhảy tới _inc

# Nếu vị trí hiện tại > vị trí gốc của input
movb (%ecx), %bl  # lấy số ở vị trí current
cmpb -1(%ecx), %bl # so sánh với vị trí current - 1
jle _dec          # nếu current <= current - 1
                  # --> nhảy tới _dec

# Nếu current > current - 1
decl %ecx         # đổi vị trí current = current - 1
jmp _for_loop     # tiếp tục vòng for

_inc:
# Xuất chuỗi "Tang dan"
movl $4, %eax
movl $1, %ebx
movl $inc_str, %ecx
movl $inc_len, %edx
int $0x80
jmp _end

_dec:
# Xuất chuỗi "Khong tang dan"
movl $4, %eax
movl $1, %ebx
movl $dec_str, %ecx
movl $dec_len, %edx
int $0x80

_end:
# Xuống dòng
movl $4, %eax
movl $1, %ebx
movl $newline, %ecx
movl $1, %edx
int $0x80

```

```
# Exit code
movl $1, %eax
int $0x80
```

Kết quả chạy thử:

```
lnghpthao@DESKTOP-463NI94:~/nt209/lab03/C31$ as -o C1.o C1.s
lnghpthao@DESKTOP-463NI94:~/nt209/lab03/C31$ ld -o C1 C1.o
lnghpthao@DESKTOP-463NI94:~/nt209/lab03/C31$ ./C1
Enter a number (4-digits): 1234
Tang dan
lnghpthao@DESKTOP-463NI94:~/nt209/lab03/C31$ ./C1
Enter a number (4-digits): 4324
Khong tang dan
lnghpthao@DESKTOP-463NI94:~/nt209/lab03/C31$ ./C1
Enter a number (4-digits): 3356
Khong tang dan
lnghpthao@DESKTOP-463NI94:~/nt209/lab03/C31$ ./C1
Enter a number (4-digits): 6893
Khong tang dan
```

Giải thích ý tưởng:

Bước 1 - Nhập số có 4 chữ số: Chương trình xuất thông báo yêu cầu người dùng nhập vào một số có 4 chữ số. Input được lưu vào vùng nhớ **input** đã được khởi tạo trước với chính xác 4 byte.

Bước 2 - Khởi tạo vị trí duyệt: Sau khi nhập, địa chỉ của chuỗi input được nạp vào thanh ghi **%ecx**. Chương trình sử dụng lệnh **leal 3(%ecx), %ecx** để dịch địa chỉ trong **%ecx**, trở trực tiếp vào chữ số thứ tư (vị trí cuối cùng của số có 4 chữ số) để bắt đầu quá trình so sánh ngược.

Bước 3 - Vòng lặp so sánh đối chiếu: Tại mỗi bước lặp, chương trình lấy ký tự tại vị trí hiện tại (lưu trong thanh ghi **%b1** với 1 byte để so sánh chính xác) và so sánh với ký tự đứng ngay trước nó (**-1(%ecx)**).

- Nếu chữ số hiện tại $\leq$ chữ số đứng trước: Dãy không tăng dần, chương trình nhảy đến nhãn **_dec** để in ra thông báo “Không tăng dần”.
- Nếu chữ số hiện tại $>$ chữ số đứng trước: Chương trình tiếp tục giảm con trỏ **%ecx** để kiểm tra cặp ký tự tiếp theo.

Bước 4 - Xác định điều kiện dừng: Vòng lặp sẽ tiếp tục cho đến khi con trỏ **%ecx** lùi về bằng hoặc nhỏ hơn địa chỉ gốc của **input**. Nếu đã duyệt qua tất cả các cặp mà không vi phạm, chương trình sẽ nhảy đến nhãn **_inc** để in ra thông báo đây là dãy "Tăng dần".

Bước 5 - Xuất kết quả và thoát: Chương trình thực hiện in chuỗi thông báo tương ứng

("Tang dan" hoặc "Khong tang dan") ra màn hình. Cuối cùng, thực hiện in ký tự xuống dòng (newline) và gọi `sys_exit` để kết thúc chương trình một cách an toàn.

C.2 Đếm số từ trong một chuỗi

Input: Chuỗi ký tự nhập từ bàn phím

Output: Xuất ra số lượng từ của một chuỗi

Giới hạn: Chuỗi 10 ký tự

Nâng cao: Xử lý chuỗi có độ dài không quá 255 ký tự, trong đó có thể có nhiều dấu cách bị lặp.

Mã nguồn (Source code):

```
.section .data
prompt_str: .ascii "Enter a string (255 chars): "
prompt_len = . - prompt_str

result_msg: .ascii "Number of words: "
result_msg_len = . - result_msg

newline: .byte 10

.section .bss
    .lcomm input, 256          # Tăng buffer lên 256 bytes
    .lcomm output_buffer, 4 #

.section .text
    .globl _start

_start:
    # 1. In ra chuỗi (sys_write)
    movl $4, %eax
    movl $1, %ebx
    movl $prompt_str, %ecx
    movl $prompt_len, %edx
    int $0x80

    # 2. Đọc chuỗi (sys_read)
    movl $3, %eax
    movl $0, %ebx
    movl $input, %ecx
    movl $256, %edx
    int $0x80
```

```

# 3. khai tao bien
movl $input, %esi      # Trỏ con trỏ vào đầu chuỗi input
movl $0, %ebx          # ebx dùng để đếm số từ (khởi tạo
= 0)
    movl $0, %ecx      # ecx là cờ (0 = ngoài từ, 1 =
trong từ)

count_loop:
    movb (%esi), %al   # Lấy 1 kí tự vào al

    cmpb $10, %al      # Gợi ý 2.3: kí tự xuống dòng
    je print_result
    cmpb $0, %al       # Hết chuỗi
    je print_result

    cmpb $32, %al      # Gợi ý 2.1: kiểm tra khoảng trắng
    je _space

    cmpl $0, %ecx      # Kiểm tra xem đang ở trong hay
ngoài chữ
    jne next_char      # Nếu đã ở trong chữ (%ecx != 0)
thì bỏ qua

    incl %ebx          # Phát hiện từ mới -> tăng biến
đếm
    movl $1, %ecx      # Bật cờ đánh dấu đang ở trong từ
    jmp next_char

_space:
    movl $0, %ecx      # Gặp khoảng trắng -> hạ cờ

next_char:
    incl %esi          # Tăng con trỏ lên kí tự kế tiếp
    jmp count_loop

print_result:
    pushl %ebx         # save kết quả đếm vào stack

    # In chu "Number of words: "
    movl $4, %eax
    movl $1, %ebx
    movl $result_msg, %ecx
    movl $result_msg_len, %edx
    int $0x80

    # Gợi ý 2.4: Chuyển đổi số nguyên thành chuỗi

```

```

    popl %eax      # lay so dem tu stack luu vao eax de chuan bi
chia
    movl $output_buffer, %edi # Trỏ edi vào buffer để chứa chữ
số
    movl $10, %esi      # Số chia là 10
    movl $0, %ecx      # Dùng ecx để đếm xem số có bao
nhiều chữ số

divide_loop:
    movl $0, %edx      # Xóa edx về 0 trước khi chia
    divl %esi          # %eax = %eax / 10, phần dư nằm ở
edx
    pushl %edx          # save phần dư vào stack
    incl %ecx          # Tăng bộ đếm chữ số
    cmpl $0, %eax      # Nếu eax còn khác 0 thì lặp lại
    jne divide_loop

    movl %ecx, %ebx      # Lưu tổng số lượng chữ số vào ebx
để làm độ dài lúc in

pop_loop:
    popl %eax          # Lấy từng chữ số từ stack ra
    addb $48, %al      # Chuyển số thành mã ascii
    movb %al, (%edi)   # Ghi vào output_buffer
    incl %edi          # Nhích buffer lên 1 ô
    loop pop_loop      # Lặp lại đến khi hết chữ số

# In con số đếm được ra màn hình
    movl $4, %eax      # sys_write
    movl %ebx, %edx     # Độ dài chuỗi số đã cất ở ebx
    movl $1, %ebx      # stdout
    movl $output_buffer, %ecx # Chỗ chứa chữ số
    int $0x80

# Xuống dòng
    movl $4, %eax
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80

# Thoát chương trình
    movl $1, %eax
    movl $0, %ebx
    int $0x80

```

Kết quả chạy thử:

```
mab@mab-VirtualBox:~$ nano c2.s
mab@mab-VirtualBox:~$ as --32 -o c2.o c2.s
mab@mab-VirtualBox:~$ ld -m elf_i386 -o c2 c2.o
mab@mab-VirtualBox:~$ ./c2
Enter a string (255 chars): i am Minh Anh
Number of words: 4
mab@mab-VirtualBox:~$ ./c2
Enter a string (255 chars): class ANTT
Number of words: 2
```

Giải thích ý tưởng:

Bước 1 - Nhập chuỗi ký tự: Chương trình xuất thông báo yêu cầu người dùng nhập vào một chuỗi ký tự (tối đa 255 ký tự). Input được hệ thống đọc thông qua ngắt `sys_read` và lưu vào vùng nhớ `input` đã được cấp phát trước đó.

Bước 2 - Khởi tạo con trỏ và biến đếm: Trước khi bắt đầu đếm, chương trình thiết lập các thanh ghi quan trọng:

- Trỏ thanh ghi `%esi` vào địa chỉ bắt đầu của chuỗi input để chuẩn bị duyệt từng ký tự.
- Khởi tạo thanh ghi `%ebx = 0` đóng vai trò là biến đếm số từ.
- Sử dụng thanh ghi `%ecx = 0` làm cờ trạng thái (flag) để nhận biết đang ở trong hay ngoài một từ: `%ecx = 0` nghĩa là đang ở ngoài từ (hoặc đứng ở khoảng trắng), `%ecx = 1` nghĩa là đang ở bên trong một từ.

Bước 3 - Vòng lặp duyệt chuỗi và đếm từ (`count_loop`): Tại mỗi bước lặp, chương trình lấy ký tự hiện tại (lưu trong thanh ghi `%al` với kích thước 1 byte) và thực hiện các kiểm tra:

- Điều kiện kết thúc: Nếu ký tự là dấu xuống dòng Enter (ASCII 10) hoặc ký tự Null (ASCII 0), vòng lặp dừng lại và chương trình nhảy đến nhãn `print_result`.
- Kiểm tra khoảng trắng: Nếu ký tự là dấu cách (ASCII 32), chương trình gán cờ `%ecx = 0` (báo hiệu đã ra khỏi từ) và chuyển sang ký tự tiếp theo. Việc hạ cờ này giúp xử lý triệt để trường hợp có nhiều dấu cách lặp lại liên tiếp mà không bị đếm sai.
- Xác định từ mới: Nếu không phải khoảng trắng (tức là chữ cái/số), chương trình kiểm tra cờ `%ecx`.
 - Nếu `%ecx = 0` (nghĩa là vừa đi từ khoảng trắng bước vào một chữ cái), đây chắc chắn là một từ mới. Chương trình sẽ tăng biến đếm `%ebx` lên 1, đồng thời bật cờ `%ecx = 1`.

- Nếu `%ecx = 1` (nghĩa là đang duyệt các chữ cái của cùng một từ), chương trình bỏ qua việc đếm và tiếp tục duyệt.

Bước 4 - Chuyển đổi số nguyên sang chuỗi (nâng cao): Để có thể in ra số lượng từ lớn hơn 9 (số có 2-3 chữ số), biến đếm `%ebx` phải được chuyển đổi từ dạng số nguyên sang chuỗi ASCII.

- Chương trình dùng vòng lặp chia liên tục số đếm cho 10 (`divl`).
- Phần dư của phép chia (chính là từng chữ số tách từ cuối lên đầu) được lưu tạm vào ngăn xếp (`pushl`).
- Sau đó, lấy ngược từng chữ số từ ngăn xếp ra (`popl`), cộng thêm 48 để chuyển thành ký tự mã ASCII và lưu lần lượt vào bộ đệm `output_buffer`.

Bước 5 - Xuất kết quả và thoát: Chương trình thực hiện in chuỗi thông báo "Number of words: " ra màn hình, tiếp nối bằng chuỗi số lượng từ đã được chuyển đổi ở Bước 4. Cuối cùng, thực hiện in ký tự xuống dòng (newline) và gọi `sys_exit` để kết thúc chương trình một cách an toàn.

Tham khảo Gemini: Do máy đang chạy hệ điều hành 64 bit, phải ép trình biên dịch chạy 32 bit để thực thi các lệnh `pushl/ popl`

Các lệnh chạy để thực thi chương trình:

```
as --32 -o c2.o c2.s
```

```
ld -m elf_i386 -o c2 c2.o
```

C.3 Chương trình đếm số chia hết cho 4 trong 5 số có 1 chữ số

Input: Nhập 5 số nguyên có 1 chữ số ($0 \leq x \leq 9$)

Output: Xuất ra màn hình số lượng số chia hết cho 4.

Mã nguồn (Source code):

```
.section .data
input_str: .string "Enter a number (1-digit): "
input_len = . -input_str

four_str: .string "Count 4x: "
four_len = . -four_str

count: .int 0
count_len = . -count
```

```

newline: .byte 10

.section .bss
    .lcomm input, 1

.section .text
    .globl _start
_start:
    # esi lưu lần lặp for, edi lưu số lượng chia hết cho 4
    xorl %esi, %esi
    xorl %esi, %edi

_for_loop:
    # Tăng số lần lặp vòng for
    incl %esi

    # So sánh số lần lặp vòng for
    cmpl $5, %esi
    jg _end

    # Xuất chuỗi yêu cầu nhập input
    movl $4, %eax           # sys_write
    movl $1, %ebx           # stdout
    movl $input_str, %ecx   # chuỗi in
    movl $input_len, %edx   # độ dài chuỗi
    int $0x80

    # Nhập chuỗi input
    movl $3, %eax           # sys_read
    movl $0, %ebx           # stdin
    movl $input, %ecx       # lưu giá trị nhập biến input
    movl $2, %edx           # độ dài tối đa của chuỗi input
    int $0x80

    # Chuyển input thành int
    xorl %eax, %eax         # clear thanh ghi eax
    movb input, %al
    subb $0x30, %al

    # Do số chia hết cho 4 sẽ có 2 bit cuối là 00
    # Thực hiện input & 3 (0011), check xem kết quả có bằng 0
    # Nếu bằng 0 --> do 11 & 00 --> số chia hết cho 4
    andb $3, %al
    jne _for_loop           # nếu kết quả khác 0

```

```

                                # --> tiếp tục vòng lặp

# Nếu kết quả bằng 0, tăng biến đếm
incl %edi
jmp _for_loop

_end:
# Xuất chuỗi "Count 4x"
movl $4, %eax
movl $1, %ebx
movl $four_str, %ecx
movl $four_len, %edx
int $0x80

# Chuyển giá trị %edi thành ASCII và chuyển vào biến count
addl $0x30, %edi
movl %edi, count

# Xuất kết quả biến đếm
movl $4, %eax
movl $1, %ebx
movl $count, %ecx
movl $count_len, %edx
int $0x80

# Xuống dòng
movl $4, %eax
movl $1, %ebx
movl $newline, %ecx
movl $1, %edx
int $0x80

# Exit code
movl $1, %eax
int $0x80

```

Kết quả chạy thử:

```

lmgphthao@DESKTOP-463NI94:~/nt209/lab03/C33$ as -o C3.o C3.s
lmgphthao@DESKTOP-463NI94:~/nt209/lab03/C33$ ld -o C3 C3.o
lmgphthao@DESKTOP-463NI94:~/nt209/lab03/C33$ ./C3
Enter a number (1-digit): 3
Enter a number (1-digit): 8
Enter a number (1-digit): 5
Enter a number (1-digit): 1
Enter a number (1-digit): 7
Count 4x: 1
lmgphthao@DESKTOP-463NI94:~/nt209/lab03/C33$ ./C3
Enter a number (1-digit): 0
Enter a number (1-digit): 8
Enter a number (1-digit): 3
Enter a number (1-digit): 2
Enter a number (1-digit): 4
Count 4x: 3
lmgphthao@DESKTOP-463NI94:~/nt209/lab03/C33$ ./C3
Enter a number (1-digit): 4
Enter a number (1-digit): 8
Enter a number (1-digit): 2
Enter a number (1-digit): 8
Enter a number (1-digit): 8
Count 4x: 4

```

Giải thích ý tưởng:

Bước 1 - Khởi tạo biến đếm: Chương trình sử dụng thanh ghi `%esi` để đếm số lần lặp (từ 1 đến 5) và thanh ghi `%edi` để lưu trữ số lượng các số chia hết cho 4 tìm thấy. Cả hai đều được reset về giá trị 0 bằng lệnh `xorl`.

Bước 2 - Vòng lặp: Tại mỗi lần lặp, chương trình xuất thông báo yêu cầu người dùng nhập một chữ số và lưu vào biến `input`. Vòng lặp sẽ dừng lại khi `%esi` vượt quá giá trị 5.

Bước 3 - Chuyển đổi ASCII sang số nguyên: Vì dữ liệu nhập từ bàn phím là ký tự ASCII, chương trình thực hiện trừ đi `0x30` (tương đương giá trị 48) từ thanh ghi `%al` để chuyển ký tự về giá trị số nguyên của nó.

Bước 4 - Kiểm tra điều kiện chia hết cho 4: Một số chia hết cho 4 khi và chỉ khi 2 bit cuối cùng của nó là `00`. Bằng cách thực hiện lệnh `andb $3, %al` (phép AND với số 3, nhị phân là `11`), nếu kết quả trả về bằng 0, số chia hết cho 4.

Bước 5 - Cập nhật kết quả:

- Nếu kết quả phép AND khác 0 (`jne`), chương trình bỏ qua và quay lại đầu vòng lặp.
- Nếu kết quả bằng 0, thanh ghi `%edi` sẽ được tăng thêm 1 đơn vị trước khi tiếp tục lần lặp tiếp theo.

Bước 6 - Xuất kết quả và thoát: Sau khi kết thúc 5 lần lặp, chương trình chuyển giá trị đếm trong `%edi` ngược lại dạng ASCII (bằng cách cộng `0x30`) và in ra màn hình cùng với chuỗi thông báo "Count 4x: ".

C.4 Kiểm tra tình trạng sinh viên dựa trên năm sinh

Input: Nhập năm sinh gồm 4 chữ số. Giả sử một SV bắt đầu học ở UIT từ 18 tuổi và theo kế hoạch sẽ tốt nghiệp lúc 22 tuổi.

Output: Xuất ra màn hình nhận định:

- + "Chua vao UIT" nếu tuổi < 18
- + "Dang hoc tai UIT" nếu tuổi từ 18 - 22
- + "Da tot nghiep" nếu tuổi > 22

Nâng cao: In thêm số tuổi, đồng thời với 3 trường hợp trên:

- + Nếu tuổi < 18, in ra năm dự kiến vào UIT.
- + Nếu tuổi 18 – 22, in ra năm dự kiến tốt nghiệp.
- + Nếu tuổi > 22, in ra năm đã tốt nghiệp.

Mã nguồn (Source code):

```
.section .data
prompt:      .string "Nhap nam sinh (4 ky tu): "  # chuỗi yêu cầu
nhập năm sinh
prompt_len = . - prompt - 1                      # độ dài chuỗi
prompt

msg_chua:    .string "Chua vao UIT\n"             # chuỗi xuất
khi tuổi < 18
msg_chua_len = . - msg_chua - 1                  # độ dài chuỗi
msg_chua

msg_dang:    .string "Dang hoc tai UIT\n"          # chuỗi xuất
khi 18 <= tuổi <= 22
msg_dang_len = . - msg_dang - 1                  # độ dài chuỗi
msg_dang

msg_da:      .string "Da tot nghiep\n"             # chuỗi xuất
khi tuổi > 22
msg_da_len = . - msg_da - 1                      # độ dài chuỗi
msg_da

current_year: .int 2026                          # năm hiện tại
dùng để tính tuổi

.section .bss
    .lcomm input, 6                             # buffer lưu năm sinh: 4 chữ số +
newline + dự phòng
```

```

.section .text
.globl _start
_start:
    # Xuất chuỗi yêu cầu nhập năm sinh
    movl $4, %eax           # sys_write
    movl $1, %ebx           # stdout
    movl $prompt, %ecx      # chuỗi in
    movl $prompt_len, %edx  # độ dài chuỗi
    int $0x80

    # Nhập năm sinh từ bàn phím
    movl $3, %eax           # sys_read
    movl $0, %ebx           # stdin
    movl $input, %ecx       # lưu giá trị nhập vào biến input
    movl $5, %edx           # đọc tối đa 5 byte (4 chữ số +
newline)
    int $0x80

    # Chuyển chuỗi 4 ký tự số sang số nguyên (năm sinh)
    # Công thức: năm = d0*1000 + d1*100 + d2*10 + d3
    movl $input, %esi       # esi trỏ đến đầu buffer input

    xorl %eax, %eax         # xóa eax về 0 (tránh giá trị rác ở
byte cao)
    movb (%esi), %al        # lấy chữ số hàng nghìn (byte 0) vào
al
    subl $48, %eax          # chuyển kí tự thành số
    movl $1000, %ecx        # hệ số nhân hàng nghìn
    imull %ecx, %eax        # eax = d0 * 1000
    movl %eax, %edi        # lưu vào edi (biến tổng tích lũy)

    xorl %eax, %eax         # xóa eax về 0
    movb 1(%esi), %al       # lấy chữ số hàng trăm (byte 1) vào
al
    subl $48, %eax          # chuyển kí tự thành số
    movl $100, %ecx         # hệ số nhân hàng trăm
    imull %ecx, %eax        # eax = d1 * 100
    addl %eax, %edi         # edi = d0*1000 + d1*100

    xorl %eax, %eax         # xóa eax về 0
    movb 2(%esi), %al       # lấy chữ số hàng chục (byte 2) vào
al
    subl $48, %eax          # chuyển kí tự thành số
    movl $10, %ecx          # hệ số nhân hàng chục
    imull %ecx, %eax        # eax = d2 * 10
    addl %eax, %edi         # edi = d0*1000 + d1*100 + d2*10

```

```

    xorl %eax, %eax                # xóa eax về 0
    movb 3(%esi), %al             # lấy chữ số hàng đơn vị (byte 3) vào
al
    subl $48, %eax                # chuyển kí tự thành
    addl %eax, %edi               # edi = năm sinh hoàn chỉnh

    # Tính tuổi = năm hiện tại - năm sinh
    movl current_year, %eax      # eax = 2026
    subl %edi, %eax              # eax = 2026 - năm sinh = tuổi

    # So sánh tuổi và rẽ nhánh
    cmpl $18, %eax               # so sánh tuổi với 18
    jl _chua_vao                 # nếu tuổi < 18 --> nhảy tới
_chua_vao

    cmpl $22, %eax               # so sánh tuổi với 22
    jg _da_tot_nghiep            # nếu tuổi > 22 --> nhảy tới
_da_tot_nghiep

    # Xuất chuỗi "Dang hoc tai UIT" (18 <= tuổi <= 22)
    movl $4, %eax                # sys_write
    movl $1, %ebx                # stdout
    movl $msg_dang, %ecx         # chuỗi in
    movl $msg_dang_len, %edx     # độ dài chuỗi
    int $0x80
    jmp _end                     # nhảy tới _end

_chua_vao:
    # Xuất chuỗi "Chua vao UIT" (tuổi < 18)
    movl $4, %eax                # sys_write
    movl $1, %ebx                # stdout
    movl $msg_chua, %ecx         # chuỗi in
    movl $msg_chua_len, %edx     # độ dài chuỗi
    int $0x80
    jmp _end                     # nhảy tới _end

_da_tot_nghiep:
    # Xuất chuỗi "Da tot nghiep" (tuổi > 22)
    movl $4, %eax                # sys_write
    movl $1, %ebx                # stdout
    movl $msg_da, %ecx           # chuỗi in
    movl $msg_da_len, %edx       # độ dài chuỗi
    int $0x80

_end:
    # Thoát chương trình

```

```
movl $1, %eax          # sys_exit
movl $0, %ebx          # exit code 0
int $0x80
```

Kết quả chạy thử (phần cơ bản):

```
mab@mab-VirtualBox:~$ nano c4.s
mab@mab-VirtualBox:~$ as --32 -o c4.o c4.s
mab@mab-VirtualBox:~$ ld -m elf_i386 -o c4 c4.o
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2006
Dang hoc tai UIT
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2004
Dang hoc tai UIT
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2003
Da tot nghiep
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2008
Dang hoc tai UIT
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2009
Chua vao UIT
```

Kết quả chạy thử (phần nâng cao):

```
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2006
Dang hoc tai UIT
Tuoi: 20
Nam du kien TN: 2028
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2004
Dang hoc tai UIT
Tuoi: 22
Nam du kien TN: 2026
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2008
Dang hoc tai UIT
Tuoi: 18
Nam du kien TN: 2030
mab@mab-VirtualBox:~$ ./c4
Nhap nam sinh (4 ky tu): 2003
Da tot nghiep
Tuoi: 23
Nam da TN: 2025
```

Giải thích ý tưởng:

Bước 1 - Khởi tạo và nhập liệu: Chương trình sử dụng các hàm hệ thống (`sys_write` và `sys_read`) để hiển thị chuỗi thông báo và đọc tối đa 5 byte dữ liệu (gồm 4 ký tự số của năm sinh và 1 ký tự xuống dòng) từ bàn phím, sau đó lưu vào vùng nhớ `input`.

Bước 2 - Chuyển đổi ASCII sang số nguyên (Năm sinh): Vì dữ liệu nhập vào là các ký tự ASCII, chương trình sử dụng thanh ghi `%esi` để trỏ đến từng byte trong vùng nhớ `input`. Tại mỗi vị trí, trừ đi 48 để lấy giá trị số nguyên thực sự. Tiếp theo, nhân các con số này với trọng số tương ứng của hàng nghìn, hàng trăm, hàng chục bằng lệnh `imull` và cộng dồn tất cả vào thanh ghi `%edi`. Kết thúc bước này, `%edi` sẽ chứa giá trị năm sinh dạng số nguyên.

Bước 3 - Tính tuổi và lưu trữ: Chương trình dùng lệnh `subl` để lấy năm hiện tại trừ đi năm sinh đang được lưu trong `%edi`. Kết quả (số tuổi) được lưu vào thanh ghi `%eax`. Đồng thời, sao chép giá trị tuổi vào biến `age_val` trong bộ nhớ `.bss` để dành cho thao tác in ấn ở cuối chương trình.

Bước 4 - Kiểm tra điều kiện và rẽ nhánh: Chương trình sử dụng lệnh `cmpl` để so sánh độ tuổi trong `%eax` với các mốc 18 và 22:

- Nếu tuổi nhỏ hơn 18 (`j1`), nhảy đến nhãn `_chua_vao`.
- Nếu tuổi lớn hơn 22 (`jg`), nhảy đến nhãn `_da_tot_nghiep`.
- Nếu không thỏa cả hai điều kiện trên, chương trình tự động đi tiếp vào phần xử lý dành cho sinh viên đang học (`18 <= tuổi <= 22`).

Bước 5 - Cập nhật dữ liệu cho từng trạng thái: Tại mỗi nhánh (đang học, chưa vào, đã tốt nghiệp), chương trình làm hai nhiệm vụ chính:

- Tính toán mốc thời gian tương ứng (năm dự kiến vào, năm dự kiến tốt nghiệp, hoặc năm đã tốt nghiệp) bằng cách cộng thêm 18 hoặc 22 vào năm sinh, rồi lưu vào biến `year_val`.
- Gán địa chỉ và độ dài của các chuỗi thông báo phù hợp (ví dụ: chuỗi "Đang học tại UIT", chuỗi "Nam du kien TN:") vào các biến nhớ tạm (`msg1_addr`, `msg2_addr`). Sau đó dùng lệnh `jmp` để nhảy thẳng đến khối lệnh in chung.

Bước 6 - Xuất kết quả chuỗi: Ở phần khối lệnh in chung (`_print_phase`), chương trình dùng `sys_write` gọi các địa chỉ chuỗi đã được gán sẵn ở Bước 5 ra màn hình. Việc thiết kế một khối in chung ở cuối giúp không phải viết lặp lại các lệnh gọi hệ thống ở từng nhánh.

Bước 7 - Chuyển đổi số nguyên sang ASCII để in (Tuổi và Năm): Để in các con số ra

màn hình, chương trình lấy dữ liệu từ `age_val` và `year_val`, dùng vòng lặp và lệnh `divl` để chia liên tục cho 10. Phần dư thu được (nằm trong `%edx`) được cộng thêm 48 để biến thành mã ASCII tương ứng và đẩy ngược từ cuối lên đầu vào một vùng bộ đệm `num_buffer`. Vòng lặp dừng lại khi thương số bằng 0.

Giải thích ý tưởng phần nâng cao:

Bước 1 - Lưu trữ thông số tạm thời : Để tránh việc phải viết lặp đi lặp lại các hàm gọi hệ thống (`sys_write`) ở từng nhánh rẽ, chương trình tạo sẵn các biến tạm trong vùng `.bss`. Khi tính xong số tuổi, thay vì in luôn, chương trình cất nó vào `age_val`.

Bước 2 - Tính toán năm tương ứng tại các nhánh: Tại mỗi nhánh điều kiện, chương trình sẽ lấy năm sinh (đang được giữ trong `%ebx`) cộng thêm các mốc tương ứng:

- Nếu chưa vào học: Năm sinh + 18 = năm dự kiến vào UIT.
- Nếu đang học hoặc đã tốt nghiệp: Năm sinh + 22 = năm dự kiến/đã tốt nghiệp. Sau khi tính xong, kết quả được lưu vào `year_val`. Đồng thời, địa chỉ của các chuỗi thông báo đi kèm (ví dụ: chuỗi "Nam du kien TN: ") cũng được trả sẵn. Sau đó dùng `jmp _print_phase` để nhảy đến khu vực in ấn chung.

Bước 3 - Thuật toán chuyển số nguyên sang chuỗi ASCII (Int to String): Để in được số tuổi hoặc số năm, chương trình phải trích xuất từng chữ số một => dùng lệnh `divl`, chương trình liên tục chia số nguyên đó cho 10.

Bước 4 - Xử lý phần dư và ghi lùi vào bộ đệm:

- Mỗi lần chia cho 10, phần dư thu được (lưu trong thanh ghi `%edx`) chính là chữ số cuối cùng của số đó (ví dụ 2026 chia 10 dư 6).
- Chương trình cộng thêm 48 vào phần dư này để biến nó thành ký tự ASCII thực sự.
- Ký tự này được nhét vào một bộ đệm `num_buffer` (có kích thước 12 byte) theo chiều từ cuối đi ngược lên đầu bằng lệnh `decl %esi` (lùi con trỏ).

Bước 5 - Điều kiện dừng và xuất chuỗi số: Vòng lặp chia ở Bước 4 sẽ lặp lại liên tục cho đến khi thương số (lưu trong `%eax`) bằng 0. Sau khi vòng lặp kết thúc, con trỏ `%esi` đang nằm ngay chữ số đầu tiên của chuỗi. Chương trình chỉ cần lấy địa chỉ cuối trừ đi địa chỉ hiện tại để ra độ dài chuỗi, và đẩy vào hàm `sys_write` để in ra màn hình.